

SYSTEM DESIGN DOCUMENT

MIRA

自分だけのパーソナル AI アシスタント

概念図と設計書 — Mac で動かし、スマホからどこでも安全に使う構成



MIRA

ゆうき専用 / 親友みたいに話す / 自分で動くエージェント

版: 1.0 / 作成日: 2026年10月8日 / 対象: 自宅の Mac(Apple Silicon, RAM 24GB)+ Galaxy S26
リポジトリ: github.com/S66Bell/AI- (ブランチ ccr-28d02a89-21mid9)

目次

1. この仕組みを一言でいうと
2. 全体像(概念図)
3. MIRA にできること
4. 構成要素の説明
5. 会話が処理される流れ
6. 記憶と学習の仕組み
7. エージェント(バックグラウンドタスクと定期実行)
8. セキュリティ設計
9. 頭脳(モデル)の選択肢
10. ファイルと設定の場所
11. 運用手順(起動・更新・困ったとき)
12. 用語集

この文書の読み方

最初の 3 章だけ読めば、仕組みの全体がつかめます。4 章以降は「中で何が起きているか」「なぜ安全か」「どう運用するか」を順に説明します。コマンドは Mac のターミナルで実行するものです。

1. この仕組みを一言でいうと

自宅の Mac の中で動く「自分専用の AI(MIRA)」に、スマホからどこでも、暗号化された専用回線でつなぐ仕組み。

会話の内容は Mac の外に出ず、料金もかからない。MIRA は話すだけでなく、検索・ファイル操作・定期的な作業をこなし、会話からあなたのことを覚えていく。

頭脳は Mac の中

オープンな言語モデル(Qwen2.5-14B)を Mac の GPU で動かす。クラウドの AI サービスは使わない(使いたい場合だけ切り替え可)。

入口はスマホのブラウザ

アプリをインストールする必要はなく、Chrome でページを開くだけ。ホーム画面に追加すれば普通のアプリのように使える。音声入力も可。

つなぐのは Tailscale

自分のアカウントでログインした端末同士だけをつなぐ暗号化ネットワーク。ポート開放なし、第三者は到達すらできない。

守りは多層

ネットワーク制限、トークン認証、ファイル操作の範囲制限、危険操作の事前確認、Web ページからの「なりすまし指示」対策など。

2. 全体像(概念図)

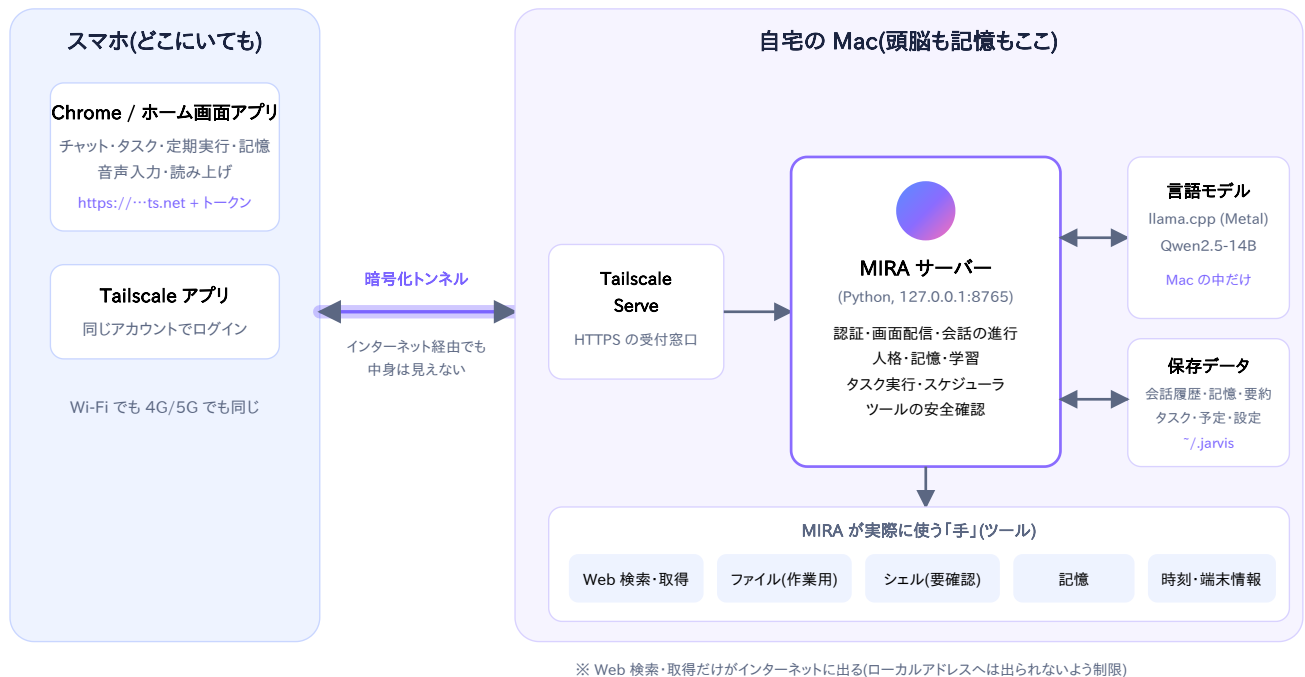


図 1: 全体像。スマホは「画面」、Mac が「頭脳・記憶・手」。間を Tailscale の暗号化トンネルがつなぐ。

ポイントは 3 つです。① 会話も記憶もモデルも Mac の中で完結する。② スマホからは HTTPS の URL を開くだけで、Mac 側には「あなたの Tailscale アカウントの端末」しか届かない。③ MIRA が外へ出るのは Web 検索・取得だけで、それも安全確認付き。

3. MIRA にできること

機能	内容	使う場所
会話	親友のようなフランクな日本語で話す。結論を先に、短く。必要ならツールを使って実際に調べる・やる。	チャットタブ
バックグラウンドタスク	「今週の AI ニュースを 3 つ調べて要約して」のような目標を渡すと、計画→ツール実行→自己チェック→報告まで一人で進める。進捗はリアルタイム、中止も可。	タスクタブ(チャットから「調べておいて」でも可)
定期実行	毎日 8:00、60 分ごと、などの予定でタスクを自動実行。Mac が寝ていた分は起動後に 1 回だけ実行。	定期実行タブ
記憶	「覚えておいて」と言われたこと、会話から読み取れる事実(名前、好み、予定、人間関係)を長期記憶に保存。古い会話は要約して保持。	記憶タブ(自動 + 手動)
音声	マイクボタンで音声入力、設定で返事の読み上げ。HTTPS 配信なのでスマホでも動く。	チャットタブ
Web 検索・ページ取得	DuckDuckGo で検索し、ページ本文を読んで答える。鍵不要・無料。	自動(MIRA が判断)
ファイル・シェル	作業用フォルダ内でファイルを読み書き、コマンドを実行。危険な操作は実行前に画面で確認。	自動(確認ダイアログ)

画面の構成

チャット

光る球体が状態表示(待機=ゆっくり、処理中=速く、エラー=赤)。「考え中… N秒」で待ち時間が分かる。■で中断。

タスク

目標を入力して「バックグラウンドで実行」。各タスクの状態(queued / running / done / failed)、報告、実行ログ。

定期実行

目標と「毎日 HH:MM」または「N 分ごと」を登録。今すぐ実行・停止・削除。

記憶

会話の要約と覚えている事実の一覧。「今の会話から学習」「履歴を全部消す」「この端末のトークンを消す」。

4. 構成要素の説明

要素	役割	実体
スマホのブラウザ(PWA)	画面。入力を送り、返事をストリーミング表示。トークンを端末内に記憶。	<code>jarvis/web/static/index.html</code>
Tailscale(スマホ・Mac)	自分のアカウントの端末だけをつなぐ暗号化ネットワーク。Mac 側の Serve 機能が HTTPS で MIRA を公開(tailnet 内のみ)。	Tailscale アプリ / <code>scripts/mac/tailscale.sh</code>
MIRA Web サーバー	認証、画面配信、API。会話を 1 つずつ処理し、ツールの確認を画面に出す。タスク・予定・学習の司令塔。	<code>jarvis/web/server.py</code>
アシスタント(人格・進行)	MIRA の人格(システムプロンプト)と記憶を組み合わせ、モデルとツールの往復(エージェントループ)を進める。	<code>jarvis/assistant.py</code> , <code>persona.py</code>
バックエンド(頭脳への接続)	どのモデルサーバーと話すか。llama.cpp / Groq / Ollama / Claude を同じ作法で扱う。小さなモデルが崩したツール呼び出しも修復。	<code>jarvis/backends/</code>
モデルサーバー	言語モデル本体を GPU で動かし、文章を生成する。Mac の中だけ(127.0.0.1)で待ち受け。	<code>llama-server</code> (Homebrew) + GGUF ファイル
ツール群	MIRA の「手」。Web 検索・取得、ファイル、シェル、記憶、時刻など。安全確認付き。	<code>jarvis/tools/</code>
記憶・学習	会話履歴、長期記憶、要約。数往復ごとに会話から事実を抽出して記憶へ。	<code>jarvis/memory.py</code> , <code>learning.py</code>
タスク実行・スケジューラ	バックグラウンドタスクを 1 つずつ実行。予定表に従って自動起動。	<code>jarvis/agent.py</code> , <code>scheduler.py</code>
設定	名前、呼び名、頭脳の種類、トークン、セキュリティの各設定。	<code>.env</code>

※ 設定項目の名前が `JARVIS_` で始まるのは、プロジェクト初期の名前の名残です。アシスタントの名前は `JARVIS_NAME` (= MIRA)で決まります。

5. 会話が処理される流れ

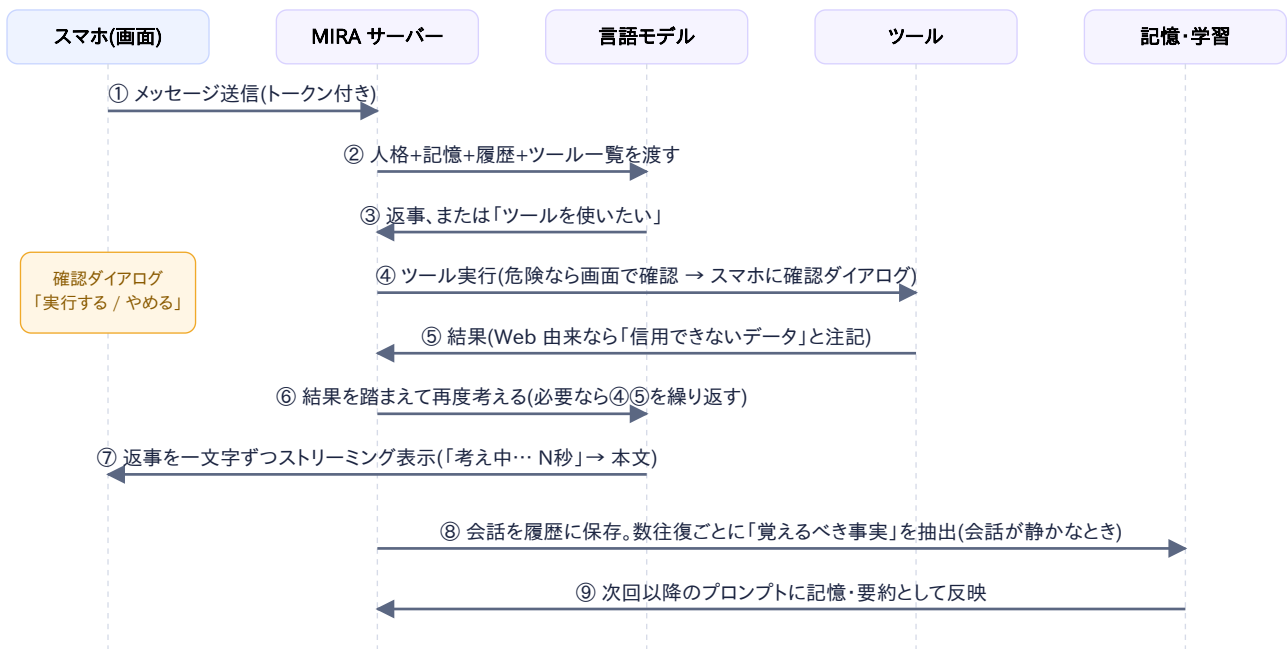


図 2: 1 回の会話の流れ。モデルは「考える」、ツールは「やる」、サーバーがその間を取り持つ。

速さを決めるもの

返事の時間は「プロンプトの読み込み」と「文章の生成」の合計です。人格・ツール一覧・記憶で約 2,000 トークンあるため、**前回の内容をキャッシュして差分だけ読む**設計にしています(時刻を分単位で入れない、学習処理は会話中に走らせない、モデルサーバーは 1 スロット)。Mac(Metal)の 14B なら、初回 5~10 秒、以降は数秒が目安です。

6. 記憶と学習の仕組み

モデル自体を学習し直す(重みを更新する)のは個人の Mac では現実的でないため、MIRA は「記憶」で学習します。会話から事実を抽出して蓄積し、毎回の会話の冒頭でそれを読み直す方式です。

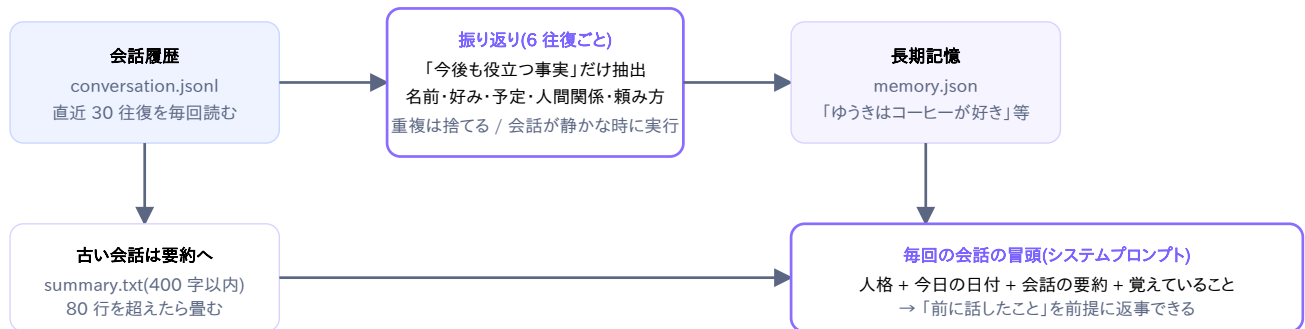


図 3: 会話 → 振り返り → 長期記憶 → 次回のプロンプト、という循環。

保存されるもの	場所	消し方
会話履歴(本文)	~/ .jarvis/conversation.jsonl	記憶タブ「履歴を全部消す」
覚えている事実	~/ .jarvis/memory.json	記憶タブで個別に「忘れる」
古い会話の要約	~/ .jarvis/summary.txt	ファイルを削除
タスク・予定	~/ .jarvis/tasks.json , schedules.json	画面から削除

7. エージェント(バックグラウンドタスクと定期実行)

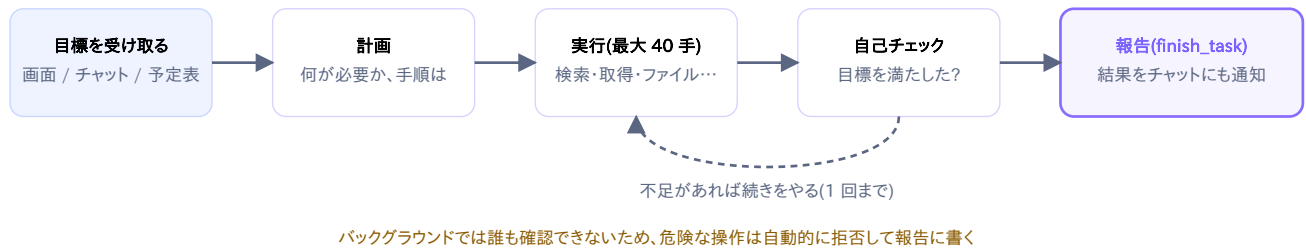


図 4: タスクは「計画 → 実行 → 自己チェック → 報告」。1 つずつ順番に処理される。

定期実行

- 「毎日 HH:MM」または「N 分ごと」で登録。20 秒ごとに予定表を確認し、時刻が来たらタスクとして実行。
- Mac がスリープしていて実行できなかった分は、起動後に **1 回だけ**まとめて実行(何回も走らない)。
- 結果は「タスク」タブに残り、チャットにも「バックグラウンドタスク完了」として通知。

チャットからの委任

「～を調べておいて」と頼むと、MIRA 自身が `start_background_task` ツールでタスクを作り、会話を止めずに裏で進めます。進捗は「～どうなった?」と聞けば答えます。

8. セキュリティ設計

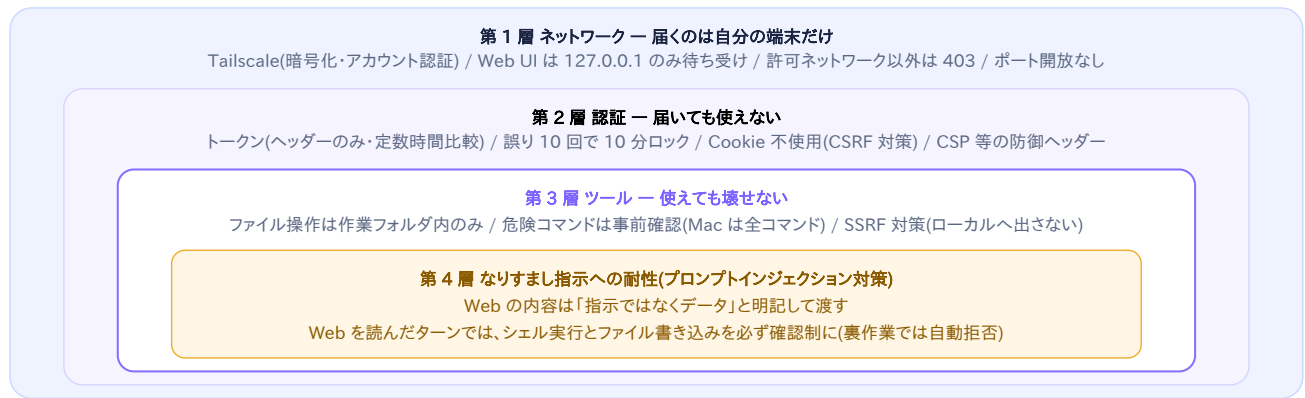


図 5: 4 層の守り。外側から順に「届かない → 使えない → 壊せない → 騙されない」。

想定する脅威	対策
第三者がインターネットから接続	Tailscale 内でしか到達できない。万一ポート開放しても許可ネットワーク外は 403。
同じ Wi-Fi の機器からの接続	Web UI は 127.0.0.1 のみ待ち受け(LAN から直接は届かない)。
トークンの総当たり	96 ビットの乱数 + 10 回失敗で 10 分ロック。
悪意あるサイト経由の操作 (CSRF)	Cookie 認証なし、JSON 以外の POST 拒否、CSP。
Web ページに仕込まれた「MIRA への指示」	信用できないデータと明記し、そのターンの副作用は確認制。裏作業は自動拒否。
AI が誤って重要ファイルを壊す	作業フォルダ外は読めない・書けない。危険コマンドは事前確認。
AI にルーターや内部サービスを触らせる	<code>web_fetch</code> はローカル/プライベートアドレスを拒否。
設定ファイル・ログからの漏えい	<code>.env</code> は 600、 <code>~/.</code> <code>jarvis</code> は 700、ログに会話本文を残さない。

残るリスク: トークンを知る人は MIRA を操作できる(人に見せない・漏れたら作り直す)。確認ダイアログは内容を読んでから押す。Groq など外部の頭脳を使うときは会話が外に出る。詳細は [docs/SECURITY.md](#)。

9. 頭脳(モデル)の選択肢

MIRA の人格・記憶・ツール・画面はそのままに、頭脳だけを切り替えられます。切り替えは `bash scripts/brain.sh ...` と再起動だけです。

頭脳	速さ(目安)	プライバシー	費用	向いている場面
既定 Mac ローカル Qwen2.5-14B	初回 5〜10 秒 以降 数秒	会話は Mac から出ない	無料	普段使い。私的な話、ファイル操作。
Groq(クラウド) Llama 3.3 70B	1〜2 秒	会話の文面が Groq に送られる(ツールは Mac 上)	無料枠(1 日 1,000 回程度)	速さと賢さ優先。長いリサーチ。
Claude(クラウド)	数秒	会話が Anthropic に送られる	有料	最高性能が必要なとき。
スマホ内(Termux)	数十秒〜	完全にスマホ内	無料	Mac が使えない非常時。実用性は低い。

Mac のモデルサイズ

RAM	自動選択	ファイル	特徴
12〜19GB	7B	約 4.7GB	速い。簡単な用途向き。
20〜39GB(今の Mac)	14B	約 9GB	賢さと速さのバランス。ツール呼び出しが安定。
40GB 以上	32B	約 20GB	かなり賢いが遅め。

モデルは Hugging Face から GGUF 形式(4bit 量子化)をダウンロードし、`~/models` に置きます。別のモデルに変えるときは `JARVIS_MODEL_PATH` で指定します。

10. ファイルと設定の場所

リポジトリ(~/.mira)

パス	内容
jarvis/	MIRA 本体(Python)。web/ が画面とサーバー、backends/ が頭脳との接続、tools/ がツール。
scripts/mac/setup.sh	初回セットアップ(llama.cpp、Python、モデル、.env)。
scripts/mac/start.sh / stop.sh	起動 / 停止。
scripts/mac/install-service.sh	ログイン時に自動起動・落ちたら再起動(LaunchAgent)。
scripts/mac/tailscale.sh	Tailscale で HTTPS 配信、LAN を閉じる。off で元に戻す。
scripts/brain.sh	頭脳の切り替え(groq キー / local)。
docs/	MAC.md(運用)、SECURITY.md(安全)、ANDROID.md(スマホ内実行)、この設計書。
.env	設定(Git には含まれない)。
tests/	自動テスト(29 件)。偽のモデルサーバーで会話・タスク・学習・安全機能を検証。

主な設定(.env)

項目	意味	今の値
JARVIS_PROVIDER	頭脳の種類(llamacpp / groq / claude / ollama)	llamacpp
JARVIS_NAME / JARVIS_USER_NAME	アシスタントの名前 / あなたの呼び名	MIRA / ゆうき
JARVIS_WEB_HOST / PORT / TOKEN	待ち受けアドレス / ポート / アクセストークン	127.0.0.1 / 8765 / 自動生成
JARVIS_WEB_ALLOW	接続を許可するネットワーク	既定(私設 + Tailscale)
JARVIS_FS_ROOT	ファイル操作の範囲	~/jarvis/workspace
JARVIS_SHELL / JARVIS_CONFIRM_ALL_SHELL	シェルツールの有無 / 全コマンド確認制	1 / 1
JARVIS_LEARN / _EVERY / _IDLE	自動学習 / 何往復ごと / 何秒静かなら	1 / 6 / 30
JARVIS_HISTORY_TURNS	起動時に読み込む過去の会話数	30
JARVIS_GROQ_API_KEY / _MODEL	Groq 使用時のキー / モデル	(未設定)

11. 運用手順

毎日の使い方

- Mac: ログインしていれば自動で動いている(常駐サービスを入れた場合)。入れていなければ `bash scripts/mac/start.sh`。
- スマホ: Tailscale をオンにして、ホーム画面の MIRA を開く。
- Mac がスリープすると届かない。ノートは電源につなぎ、電源接続時のスリープを防ぐ設定にする。

よく使うコマンド

```
cd ~/mira
bash scripts/mac/start.sh           # 起動(Ctrl-C で停止)
bash scripts/mac/stop.sh            # 停止(常駐サービスも止める)
bash scripts/mac/install-service.sh # ログイン時に自動起動(remove で解除)
git pull                            # 最新版に更新(そのあと再起動)
bash scripts/brain.sh groq gsk_キー # 頭脳を Groq に
bash scripts/brain.sh local         # 頭脳を Mac に戻す
bash scripts/mac/tailscale.sh off   # Tailscale 配信をやめて LAN に戻す
```

トークンを作り直す

```
sed -i '' "s|^JARVIS_WEB_TOKEN=.*|JARVIS_WEB_TOKEN=$(.venv/bin/python -c 'import secrets;print(secrets.token_hex(12))')|\" .env
grep JARVIS_WEB_TOKEN .env          # 新しいトークンを確認 → 再起動 → スマホで再入力
```

困ったとき

症状	確認・対処
スマホで「このサイトにアクセスできません」	Mac で MIRA が動いているか(<code>start.sh</code> の画面)。スマホの Tailscale がオンか。Mac がスリープしていないか。
トークンを聞かれ続ける	<code>.env</code> の <code>JARVIS_WEB_TOKEN</code> を確認。10 回間違えると 10 分ロック。
返事が遅い	<code>grep -E "prompt eval time eval time" ~/.jarvis/llama-server.log tail -n 4</code> で速度を確認。他の重いアプリを閉じる。7B に変える。
起動時に <code>llama-server exited</code>	<code>tail -n 30 ~/.jarvis/llama-server.log</code> 。メモリ不足なら小さいモデルへ。
中国語や英語で返してくる	人格の指示で日本語を指定済み。続いたら「日本語で」と一言。14B ではほぼ起きない。
常駐サービスのログを見たい	<code>tail -f ~/.jarvis/mac.log</code> 、会話処理の記録は <code>~/.jarvis/web.log</code> 。

12. 用語集

用語	意味
言語モデル(LLM)	文章を読んで続きを生成する AI の本体。MIRA の「頭脳」。
GGUF / 量子化	モデルを小さく圧縮した形式。4bit 量子化で品質をほぼ保ったままサイズを 1/4 に。
llama.cpp / llama-server	GGUF モデルを Mac などで高速に動かすソフト。Metal(Apple の GPU 機能)を使う。
トークン	モデルが扱う文字のかたまり。日本語は 1~2 文字で 1 トークン程度。「トークン/秒」が速さの単位。
プロンプト / システムプロンプト	モデルに渡す文章。冒頭の「人格・記憶・ルール」部分がシステムプロンプト。
ツール呼び出し	モデルが「検索して」「このファイルを読んで」と要求し、サーバーが実行して結果を返す仕組み。
エージェント	目標に向けてツールを何度も使い、自分で進める AI の使い方。
PWA	ブラウザのページをアプリのようにホーム画面に置ける仕組み。HTTPS が必要。
Tailscale / tailnet	自分の端末同士だけをつなぐ暗号化ネットワーク(VPN の一種)。その自分専用ネットワークが tailnet。
Tailscale Serve	tailnet 内の端末に対して、Mac 上のサービスを HTTPS で公開する機能。
アクセストークン	MIRA を使うための合言葉。ブラウザが記憶し、毎回ヘッダーで送る。
プロンプトインジェクション	Web ページなどに「AI への偽の指示」を仕込み、AI を操ろうとする攻撃。
SSRF	サーバー(ここでは MIRA)に、内部ネットワークへアクセスさせる攻撃。
CSP / CSRF	ブラウザ側の防御設定 / 他サイト経由で操作させる攻撃。MIRA は両方に対策済み。
KV キャッシュ	モデルが一度読んだプロンプトの計算結果。使い回すと 2 回目以降が速くなる。

この設計書は MIRA のソースコードと一緒に `docs/design/` に保存されています。仕組みを変更したら、この文書も更新してください。